

Week 8: Streamlit and Web App Dev



MAISI



Building Interpretable AI in Healthcare
Michigan Data Science Team - Winter 2026



Week 8 Agenda

01.

Icebreaker!

Every great project work session starts off with a great icebreaker.

02.

Intro to Streamlit

What is Streamlit, how does it make web app development easy, and what are some functions you need to know?

03.

Final Project Overview

What are our goals for your final demo, and how can we get there?

04.

Hands-On Data Science!

Let's begin building your website to showcase all of the awesome data science skills you have learned this semester!

01 Fun Icebreaker!!

Get to know us and start to learn about
each other with a fun icebreaker!



Icebreaker – Week 8

Share with the people around you!! :)

What is your dream vacation destination?





02

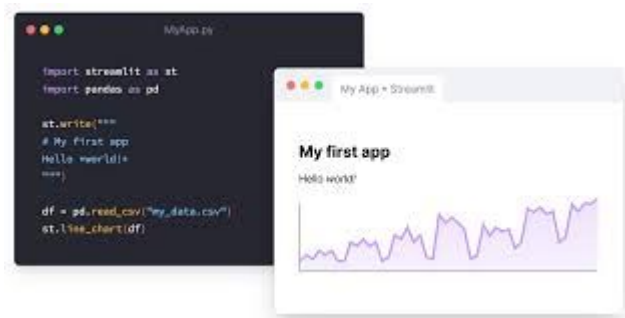
Intro to Streamlit

What is Streamlit, how does it make web app development easy, and what are some functions you need to know?



What is Streamlit?

- **Streamlit** is an open-source Python framework for creating *interactive* data apps.
 - Directly written in Python syntax, so the commands/structure will be familiar
- No need for extensive front-end (user-facing) development knowledge, as that is fully handled with simple, backend-style commands
 - Your main focus with Streamlit will be integrating model functionality into digestible, interactive widgets





Why is Streamlit Useful?

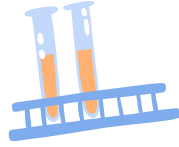
- Streamlit significantly lowers the barrier to entry for web app development, especially compared to learning something like HTML.
 - Because it's in Python syntax, all you really need to know is a few functions!
- A web app is a highly-desirable deliverable to put on a LinkedIn profile, send to interviewers, or show to your friends and family.
 - Great way to bring a ton of disjointed Jupyter notebooks into one, clean compilation of everything you have learned.
 - Boosts interpretability by allowing for better formatting, digestible structure, and thorough explanations of your findings.



03

Final Project Overview

What are our goals for your final demo, and how can we get there?





What is the Goal for this Project?

- So far, you all have done a ton of great work learning about every step of the data analysis and the machine learning processes:
 - Choosing a Good Dataset
 - Exploratory Data Analysis (Visualization, Data Cleaning, Feature Engineering)
 - Machine Learning (Different Models, Train/Test Splits, Optimization)
 - Uncertainty Quantification (MAPIE, SHAP)
- Our goal: give you all the tools to show off these skills in an easy-to-implement yet attractive way!
 - The answer: A Python-based web app built using Streamlit!



What will this Web App Contain?

- Home Page
 - Summarizes the rest of the pages, brief overview of your model/findings
- Data Overview/Analysis Page
 - Explains the origins and contents of your dataset, including thorough explanations of what each column/feature represents
 - Then, either on the same page or a different one, showcase your EDA and visualizations
- Model Overview/Analysis Page
 - Detail which model you chose, the final features you selected to train it, and some of its performance metrics
- Interactive Predictions and Uncertainty Quantification
 - Here is where you're going to capitalize on the ability to create interactive widgets with Streamlit!
Allow the user to adjust the features of a hypothetical patient, make a prediction, and run MAPIE and SHAP to fully explain where those decisions came from



0

4

References

Here are a handful of useful guides to high-utility functions, Git demos, and Colab explanations!

Our Demo! (for reference)



Link:

<https://mdst-ai-in-healthcare.streamlit.app/>

Project Timeline

Week 1 (01/25): Icebreaker/EDA intro, choosing a good dataset

Week 2 (02/01): Data Visualization, Further EDA

Week 3 (02/08): Feature Engineering and Intro to Machine Learning

Week 4 (02/15): Machine Learning and Model Optimization

Week 5 (02/22): Conformal Predictions, Risk Control, and Multi-Class Calibration

NO MEETING 03/01 OR 03/08 – SPRING BREAK

Week 6 (03/15): SHAP Values and Feature Importance Interpretation

Week 7 (03/22): Intro to Streamlit and Project Work Time

Weeks 8 + 9 (03/29 + 04/05): Final Project work time + Data Science Night prep!



Important Functions to Know (pt. 1)

Syntax	Description
<code>import pandas as pd</code>	Import the pandas library, using the alias pd for convenience
<code>import numpy as np</code>	Import the NumPy library, using alias np for numerical operations
<code>import seaborn as sns</code>	Import Seaborn for statistical data visualization
<code>df = pd.read_csv('file.csv')</code>	Load a CSV file into a pandas DataFrame for data manipulation
<code>df.head()</code>	Returns the first 5 rows of the DataFrame, useful for quickly inspecting data
<code>df.info()</code>	Provides a concise summary of the DataFrame, including data types and non-null values

Important Functions to Know (pt. 2)

Syntax	Description
<code>df.describe()</code>	Generates descriptive statistics of numerical columns (mean, median, quartiles, etc.)
<code>df['column_name']</code>	Access a specific column in the DataFrame, works like a key in a dictionary
<code>df.drop(columns=['col'...])</code>	Drops specified columns from the DataFrame, use <code>inplace=True</code> to modify the original DataFrame
<code>df.index</code>	Returns the index labels of the DataFrame
<code>df.isnull()</code>	returns a DataFrame of boolean values , where each entry indicates whether the corresponding value in df is NaN (missing)

Important Functions to Know (pt. 3)

Syntax	Description
<code>df['column'].value_counts()</code>	Returns the count of unique values in a specific column
<code>df['column'].mean()</code>	Returns the mean value of a numerical column
<code>df.corr()</code>	Computes correlation for numerical columns to understand relationships
<code>df.columns</code>	Lists all column names in the DataFrame , useful for renaming or viewing dataset structure
<code>df.shape</code>	Returns # of rows and columns

Important Functions to Know (pt. 4)

Syntax	Description
<code>df.rename(columns={'old' : 'new'}, inplace=True)</code>	Renames specific columns to new names
<code>df.groupby('category')['value']</code>	Groups the DataFrame using a specified column to perform aggregate functions (e.g., <code>.sum()</code> , <code>.mean()</code>)
<code>df['new_column'] = df['column'].apply(function)</code>	Applies a function to each element or column/row
<code>df.shape</code>	Returns a tuple representing the dimensions (rows, columns) of the DataFrame
<code>df.dropna()</code>	Removes rows or columns containing missing values

Important Functions to Know (pt. 5)

Syntax	Description
<code>df.to_numpy()</code>	Converts Pandas DataFrame into a NumPy array
<code>np.shape</code>	Returns a tuple representing the dimensions (rows, columns) of the array (similar to <code>df.shape</code>)
<code>np.reshape()</code>	Reshapes an original numpy array to the specified dimensions (rows, columns)
<code>np.array()</code>	Creates an N-dimensional array (ndarray) which is more memory efficient than standard python lists
<code>np.unique()</code>	Returns the unique elements in a NumPy array (also returns the counts of each element given the keyword argument, <code>return_counts=True</code>)

Important Functions to Know (pt. 6)

Syntax	Description
<code>pd.cut()</code>	Turns continuous numerical data into categorical data
<code>pd.get_dummies(drop_first = True)</code>	Dummy encodes categorical data (used with <code>drop_first</code> argument)
<code>OrdinalEncoder()</code>	Function for encoding categorical information that has an order
<code>train_test_split(X, y, test_size=0.2, random_state=42)</code>	Splits data into a training set and a testing set . The test set size can be changed
<code>LinearRegression()</code>	Used for initializing a linear regression model
<code>LogisticRegression()</code>	Used for initializing a logistic regression model

Important Functions to Know (pt. 7)

Syntax	Description
StandardScaler()	Turns continuous numerical data into categorical data
model.fit(X_train, y_train)	Fits a model to the training data
model.predict(X_test)	Uses the test data to make predictions
.coef_	Attribute for getting the models coefficients (for linear and logistic regression)
mean_squared_error()	Calculates the average squared distance between the predicted and actual data points
mean_absolute_error()	Calculates the average distance between the predicted and actual values

How to Upload Files into Colab

Click on the folder in the sidebar



The screenshot shows a Google Colab interface. On the left, the 'Files' sidebar is open, showing a folder named 'sample_data'. A red arrow points to this folder from the text 'Click on the folder in the sidebar'. The main notebook area is titled 'Week 1 - Pandas Practice'. It contains three code cells:

- Cell 1: Imports pandas and seaborn, and mounts Google Drive.

```
import pandas as pd
import seaborn as sns

# Allows you to provide a path to a Google Drive address rather than a local file path
from google.colab import drive
drive.mount('/content/drive')
```

Below the code, it says 'Mounted at /content/drive'.
- Cell 2: Loads a CSV file into a DataFrame.

```
df = pd.read_csv('/content/MDST Week 1 - Pandas Practice.csv')
```
- Cell 3: Prints the first five rows and datatypes.

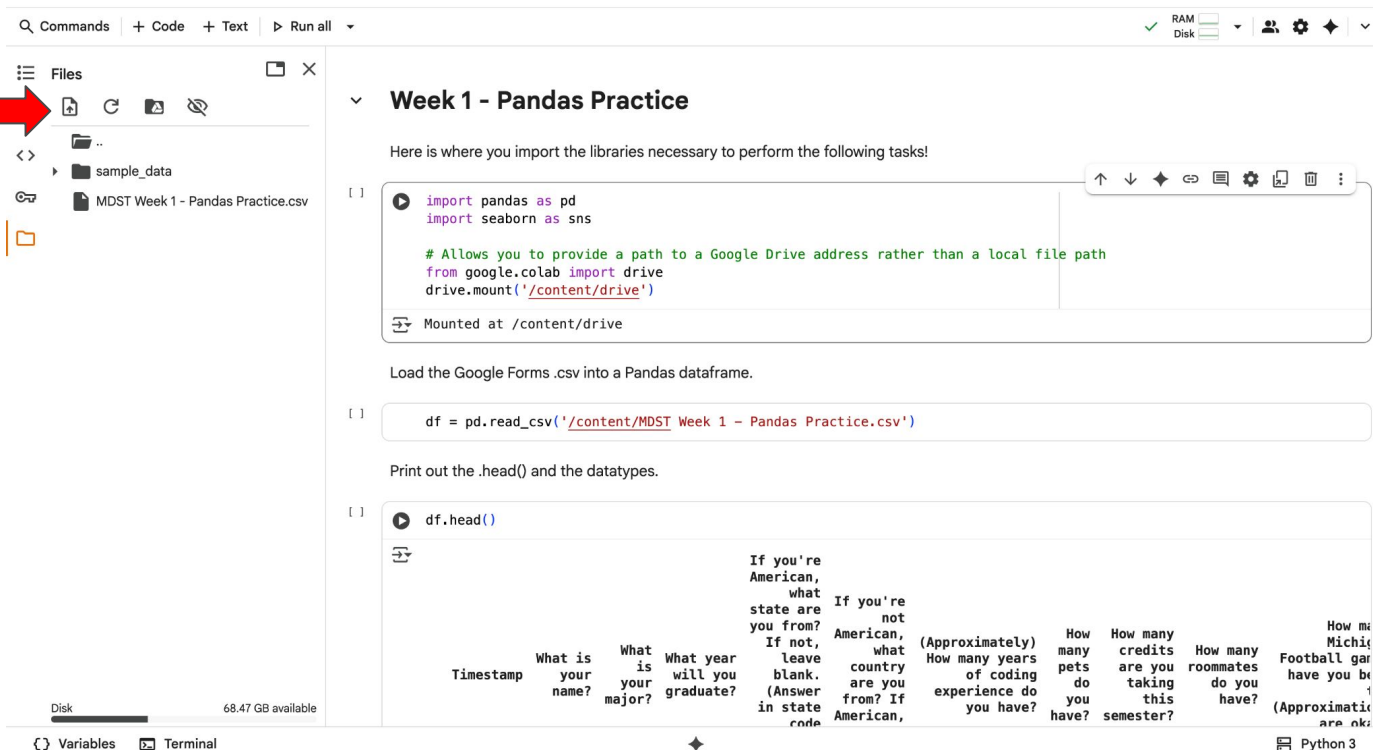
```
df.head()
```

The output of the third cell shows a preview of the data with columns: Timestamp, What is your name?, What is your major?, What year will you graduate?, If you're American, what state are you from?, If not, leave blank. (Answer in state code), If you're American, what country are you from? If American, (Approximately) How many years of coding experience do you have?, How many pets do you have?, How many credits are you taking this semester?, How many roommates do you have?, and How many football games have you been to? (Approximate are n...

At the bottom, the status bar shows 'Disk' with '68.47 GB available', 'Variables', 'Terminal', and 'Python 3'.

How to Upload Files into Colab

Click the upload button and select the file you want to upload



The screenshot shows the Google Colab interface. On the left, the 'Files' pane displays a directory structure with a folder named 'sample_data' and a file named 'MDST Week 1 - Pandas Practice.csv'. A red arrow points to the upload button (a square with a plus sign) in the file manager. The main area is titled 'Week 1 - Pandas Practice' and contains the following text: 'Here is where you import the libraries necessary to perform the following tasks!'. Below this, a code cell is shown with the following Python code:

```
import pandas as pd
import seaborn as sns

# Allows you to provide a path to a Google Drive address rather than a local file path
from google.colab import drive
drive.mount('/content/drive')
```

The output of the code cell shows 'Mounted at /content/drive'. Below the code cell, the text 'Load the Google Forms .csv into a Pandas dataframe.' is displayed. A second code cell contains the following Python code:

```
df = pd.read_csv('/content/MDST Week 1 - Pandas Practice.csv')
```

The output of this code cell shows the first few rows of the dataframe:

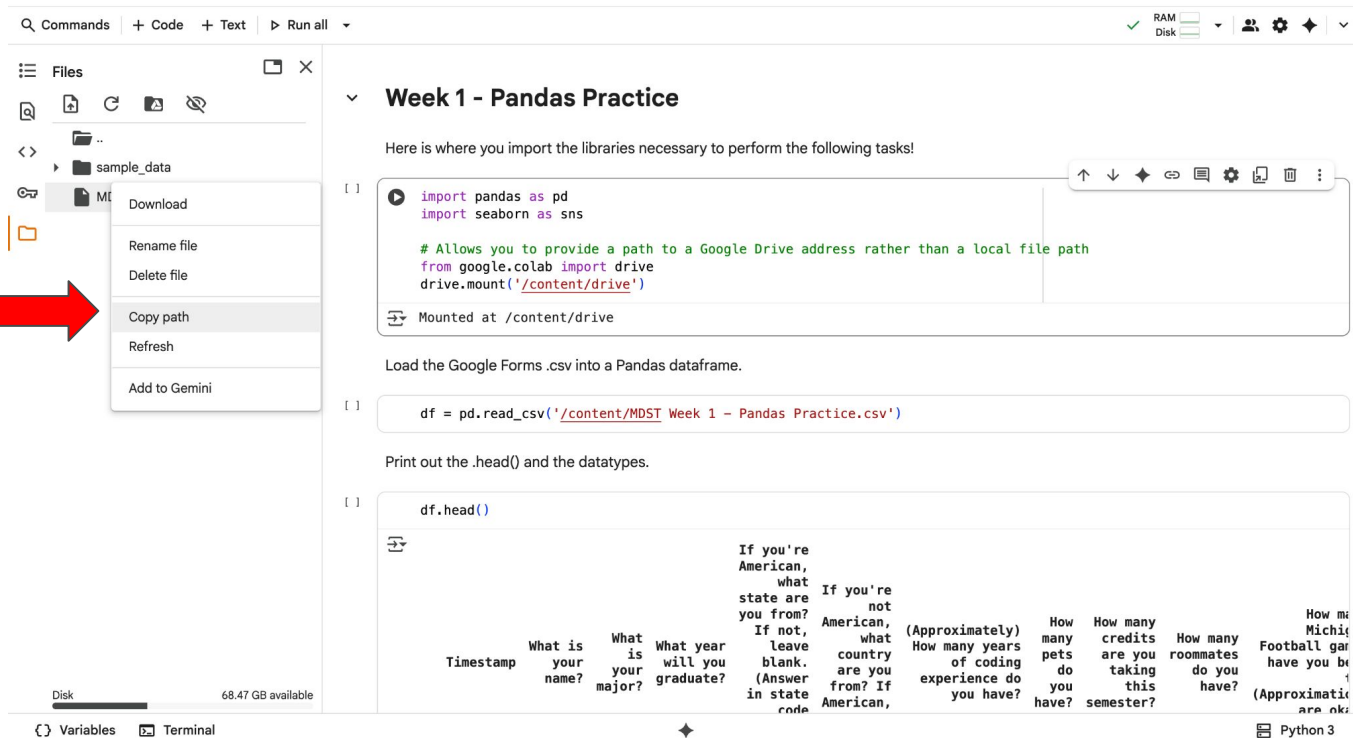
```
df.head()
```

Timestamp	What is your name?	What is your major?	What year will you graduate?	If you're American, what state are you from?	If you're not American, what country are you from?	If not, leave blank. (Answer in state code)	How many years of coding experience do you have?	How many pets do you have?	How many credits are you taking this semester?	How many roommates do you have?	How many Football games have you been to?	How many Michigan are you from?

The bottom of the interface shows the 'Variables' pane with 'df' and the 'Terminal' pane.

How to Upload Files into Colab

Click the three dots and copy the path. Put this in your read function



The screenshot shows the Google Colab interface. On the left, the 'Files' pane displays a folder named 'sample_data' containing a file 'MDST Week 1 - Pandas Practice.csv'. A context menu is open for this file, with the 'Copy path' option highlighted. A red arrow points from the text instruction to this option. The main area shows the code for mounting Google Drive and reading the CSV file into a Pandas dataframe.

```
import pandas as pd
import seaborn as sns

# Allows you to provide a path to a Google Drive address rather than a local file path
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

Load the Google Forms .csv into a Pandas dataframe.

df = pd.read_csv('/content/MDST Week 1 - Pandas Practice.csv')

Print out the .head() and the datatypes.

df.head()
```

The output of `df.head()` is a table with columns: Timestamp, What is your name?, What is your major?, What year will you graduate?, If you're American, what state are you from?, If not, leave blank. (Answer in state code), If you're American, what country are you from? If American, (Approximately) How many years of coding experience do you have?, How many pets do you have?, How many credits are you taking this semester?, How many roommates do you have?, How many Football games have you been to? (Approximately) How many are ok?

How to Mount Your Google Drive

```
from google.colab import drive

drive.mount('/content/drive')

pd.read_csv('/content/drive/MyDrive/[FILE NAME]')
```

****NOTE:** If you saved your dataset in a folder (not just loose in your MyDrive folder), you will need to add further code to the file path before the file name in the final line. For example, if I saved my data called “alzheimers.csv” in a folder called “MDST-W26,” my read_csv statement would read:

```
pd.read_csv('/content/drive/MyDrive/MDST-W26/alzheimers.csv')
```